

Francisco Javier Pirir González
201902227

1. Estructuras de Datos Eficientes

Árboles Binarios de Búsqueda (ABB)

Un Árbol Binario de Búsqueda, también conocido como ABB, es una estructura de datos organizada en forma de árbol. Cada nodo puede tener como máximo dos hijos: un hijo izquierdo y un hijo derecho.

La regla principal de ordenamiento es la siguiente:

- Los valores menores que el nodo principal se colocan en el subárbol izquierdo.
- Los valores mayores que el nodo principal se colocan en el subárbol derecho.

Por ejemplo, si el nodo raíz tiene el valor 10, un valor como 5 se colocaría a la izquierda, mientras que un valor como 15 se colocaría a la derecha.

La principal desventaja de un ABB ocurre cuando los datos se insertan en orden secuencial, por ejemplo: 1, 2, 3, 4, 5. En este caso, el árbol pierde su forma balanceada y se convierte prácticamente en una lista enlazada. Esto hace que las operaciones de búsqueda, inserción y eliminación sean menos eficientes, llegando a una complejidad de $O(n)$.

Árboles AVL

Un Árbol AVL es un árbol binario de búsqueda auto-balanceado. Esto significa que el árbol se reorganiza automáticamente cuando se insertan o eliminan nodos, con el objetivo de mantener una altura equilibrada.

El balance del árbol se controla por medio del factor de balanceo:

Factor de balanceo = Altura izquierda - Altura derecha

En un árbol AVL, el factor de balanceo de cada nodo debe mantenerse entre -1, 0 y 1. Si el factor de balanceo se sale de ese rango, el árbol realiza rotaciones para recuperar el equilibrio.

Gracias a este balance automático, las operaciones de búsqueda, inserción y eliminación mantienen una complejidad de $O(\log n)$. Esto ocurre porque la altura del árbol se mantiene controlada, evitando que se degenere en una lista enlazada.

2. Fundamentos de Web APIs

¿Qué es una API?

Una API, o Interfaz de Programación de Aplicaciones, es un conjunto de reglas que permite que diferentes aplicaciones se comuniquen entre sí. En el caso de una Web API, esta comunicación se realiza por medio de internet o una red, utilizando el protocolo HTTP.

Por ejemplo, una aplicación cliente puede solicitar datos a una API, y la API responde con la información solicitada en un formato como JSON.

Modelo Cliente-Servidor

El modelo cliente-servidor es una forma de comunicación donde existen dos partes principales:

El cliente es quien realiza una solicitud. Puede ser un navegador web, una aplicación móvil, Postman, REST Client o cualquier programa que consuma la API.

El servidor es quien recibe la solicitud, procesa la información y devuelve una respuesta.

El proceso funciona así:

1. El cliente envía una petición HTTP al servidor.
2. El servidor recibe la petición.
3. El servidor procesa la solicitud.
4. El servidor devuelve una respuesta HTTP.
5. El cliente recibe y muestra la información.

Por ejemplo, cuando se accede a una ruta como `/api/nodos`, el navegador actúa como cliente y la API creada en ASP.NET Core actúa como servidor.

3. Verbos HTTP

GET

El verbo HTTP GET se utiliza para recuperar o consultar recursos del servidor. No debe modificar la información almacenada.

Este método devuelve una lista de nodos en formato JSON.

GET es idempotente, lo que significa que, si se ejecuta varias veces, el resultado será el mismo mientras los datos no cambien. Es decir, consultar la lista de nodos no altera la información almacenada.

POST

El verbo HTTP POST se utiliza para crear nuevos recursos en el servidor. A diferencia de GET, POST sí puede modificar la información, porque agrega nuevos datos.

POST no es idempotente, porque si se ejecuta varias veces la misma petición, podría crear varios recursos repetidos o modificar el estado del servidor varias veces.

Capturas

```
1  var builder = WebApplication.CreateBuilder(args);
2  var app = builder.Build();
3
4  // Base de datos simulada en memoria
5  var coleccionNodos = new List<NodoElemento>
6  {
7      new NodoElemento { Id = 10, Valor = "Raíz Inicial (ABB)" },
8      new NodoElemento { Id = 5, Valor = "Hijo Izquierdo" }
9  };
10
11 // GET: Retorna todos los nodos actuales
12 app.MapGet("/api/nodos", () =>
13 {
14     return Results.Ok(coleccionNodos);
15 });
16
17 // POST: Recibe un nuevo nodo y lo inserta en la colección
18 app.MapPost("/api/nodos", (NodoElemento nuevoNodo) =>
19 {
20     if (nuevoNodo.Id <= 0 || string.IsNullOrEmpty(nuevoNodo.Valor))
21     {
22         return Results.BadRequest("Datos del nodo inválidos.");
23     }
24
25     coleccionNodos.Add(nuevoNodo);
26
27     return Results.Created($"/api/nodos/{nuevoNodo.Id}", nuevoNodo);
28 });
29
30 app.Run();
```

```
4 references
✓ public class NodoElemento
{
    4 references
    public int Id { get; set; }
    3 references
    public string Valor { get; set; } = string.Empty;
}
```

```
PS C:\Users\Dell\Documents\IPC 2 Vacas\acti11_06\ApiEstructurasDemo> dotnet run
Usando la configuración de inicio de C:\Users\Dell\Documents\IPC 2 Vacas\acti11_06\ApiEstructurasDemo\Properties\launch
Settings.json...
Compilando...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5241
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\Dell\Documents\IPC 2 Vacas\acti11_06\ApiEstructurasDemo
info: Microsoft.Hosting.Lifetime[0]
      Application is shutting down...
```

```
1  HTTP/1.1 201 Created
2  Connection: close
3  Content-Type: application/json; charset=utf-8
4  Date: Fri, 12 Jun 2026 00:32:34 GMT
5  Server: Kestrel
6  Location: /api/nodos/15
7  Transfer-Encoding: chunked
8
9  {
10   "id": 15,
11   "valor": "Nuevo Nodo Derecho"
12 }
```

← → ↻ 🏠 ⓘ localhost:5241/api/nodos

Dar formato al texto ☒

```
[
  {
    "id": 10,
    "valor": "Raíz Inicial (ABB)"
  },
  {
    "id": 5,
    "valor": "Hijo Izquierdo"
  },
  {
    "id": 15,
    "valor": "Nuevo Nodo Derecho"
  }
]
```

Repositorio

https://github.com/panxo90/actividad11_06.git